

POCO4A

PROGRAMAÇÃO ORIENTADA

A OBJETOS

Aula 03 - Construtores e Destrutores

Prof. Rafael G. **Mantovani**

Roteiro



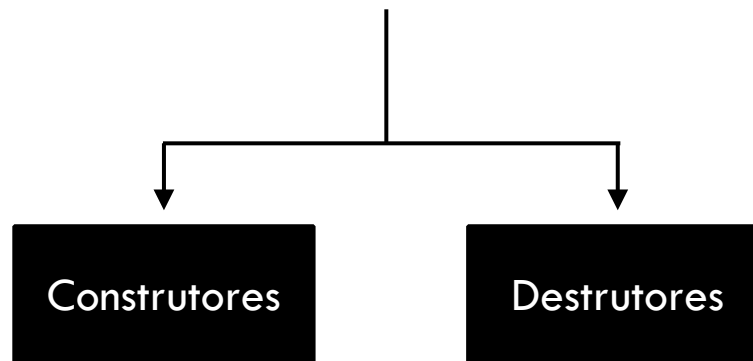
- 1** Introdução
- 2** Construtores
- 3** Destrutores
- 4** Exemplos e Exercícios
- 5** Referências

Roteiro

- 1** Introdução
- 2** Construtores
- 3** Destrutores
- 4** Exemplos e Exercícios
- 5** Referências

Introdução

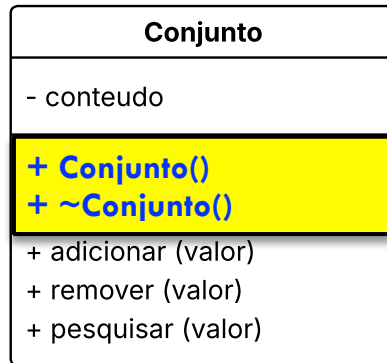
Métodos Especiais



Introdução

Conjunto
- conteudo
+ Conjunto () + ~ Conjunto () + adicionar (valor) + remover (valor) + pesquisar (valor)

Introdução



Construtor: inicializa o objeto



Destruitor: finaliza o objeto

Roteiro



- 1 Introdução
- 2 Construtores**
- 3 Destrutores
- 4 Exemplos e Exercícios
- 5 Referências

Construtores

- Um **Construtor** é uma função membro especial que é chamada automaticamente quando um objeto da classe é criado. Seu objetivo primário é **inicializar os atributos do objeto** e estabelecer um estado inicial válido.

Construtores

- Um **Construtor** é uma função membro especial que é chamada automaticamente quando um objeto da classe é criado. Seu objetivo primário é **inicializar os atributos do objeto** e estabelecer um estado inicial válido.

- Mesmo nome da classe (**C++**), `__init__` (**Python**)
- **Sem** tipo de retorno
- **Invocação Automática**
- Acessibilidade (**públicos**), mas podem ser privados/protegidos (singleton)
- Sem herança

Construtores

- **Tipos de Construtores**



Default

Construtores

- **Tipos de Construtores**



Default : sem parâmetros or com valores predefinidos

Construtores



C++

```
1. class Time{
2.
3.     public:
4.         Time();
5.         void setTime(int, int, int);
6.         void printUniversal();
7.         void printStandard();
8.
9.     private:
10.         int hour;    // 0 - 23 (formato de relógio de 24 horas)
11.         int minute;  // 0 - 59
12.         int second;  // 0 - 59
13. };
```

Construtores

□ Tipos de Construtores

 **Default** : sem parâmetros or com valores predefinidos



```
14. Time::Time() {  
15.     hour = minute = second = 0;  
16. }
```

Construtores

□ Tipos de Construtores

△ **Default** : sem parâmetros or com valores predefinidos



```
14. Time::Time() {  
15.     hour = minute = second = 0;  
16. }
```



```
14. Time::Time():hour(0),minute(0),second(0){  
15.  
16. }
```

Construtores

□ Tipos de Construtores

△ **Default** : sem parâmetros or com valores predefinidos

C++

```
14. Time::Time() {  
15.     hour = 0;  
16. }
```

□ C++: compilador gera um construtor vazio se nenhum for definido

C++

```
14. Time::Time():hour(0),minute(0),second(0){  
15.  
16. }
```

Construtores

- **Tipos de Construtores**

- △ **Parametrizado** : recebe parâmetros para definir os atributos

Construtores

□ Tipos de Construtores

△ **Parametrizado** : recebe parâmetros para definir os atributos



```
17. Time::Time(int h, int m, int s) {  
18.     hour    = (h >= 0 && h < 24) ? h:0;  
19.     minute  = (m >= 0 && m < 60) ? m:0;  
20.     second  = (s >= 0 && s < 60) ? s:0;  
21. }
```

Construtores

□ Tipos de Construtores



Cópia: copia um objeto inteiro sem mudar o objeto copiado

Construtores

□ Tipos de Construtores



Cópia: copia um objeto inteiro sem mudar o objeto copiado



C++

```
22. Time::Time(const Time &other) {  
23.     hour    = other.hour;  
24.     minute  = other.minute;  
25.     second  = other.second;  
25. }
```

Construtores

□ Tipos de Construtores



Cópia: copia um objeto inteiro sem mudar o objeto copiado



C++

```
22. Time::Time(const Time &other) {  
23.     hour    = other.hour;  
24.     minute  = other.minute;  
25.     second  = other.second;  
25. }
```

Obs: Implícito desde o C++ 11

Roteiro



- 1 Introdução
- 2 Construtores
- 3 Destrutores
- 4 Exemplos e Exercícios
- 5 Referências

Destrutores

- Um **Destrutor** é uma função membro especial que é chamada automaticamente quando um objeto da classe é **destruído**. Isso ocorre, logo que a execução do programa deixa o escopo em que esse objeto foi instanciado

Destrutores

- Um **Destrutor** é uma função membro especial que é chamada automaticamente quando um objeto da classe é **destruído**. Isso ocorre, logo que a execução do programa deixa o escopo em que esse objeto foi instanciado

- Mesmo nome da classe precedido por ~ (**C++**),
__del__ (**Python**)
- **Sem** tipo de **retorno** e **sem** **parâmetros**
- O destrutor em si não libera a memória do objeto, ele faz a **faxina de terminação** antes de o sistema reivindicar a memória do objeto
- **Único** destrutor por classe, sem sobrecarga
- Toda classe tem um destrutor, se o programador não explicita, o compilador cria um destrutor vazio

Destrutores



C++

Destrutores



```
1. #include<string>
2. #include<iostream>
3.
4. using namespace std;
5.
6. class CreateAndDestroy {
7.
8.     public:
9.         CreateAndDestroy(int, string); //construtor
10.        ~CreateAndDestroy();           //destrutor
11.
12.     private:
13.         int objectID;    //numero de ID do objeto
14.         string message; //mensagem descrevendo o objeto
15. };
```

Destrutores



```
16. // Implementação do Construtor
17. CreateAndDestroy::CreateAndDestroy(int ID, string
18. messageString) {
19.     objectID = ID;
20.     message = messageString;
21.     cout << "Object " << objectID << " constructor runs" <<
22.     message << endl;
23. }
24.
25. // Implementação do Destrutor
26. CreateAndDestroy::~CreateAndDestroy() {
27.     cout << "Object " << objectID << " destructor runs " <<
28.     message << endl;
29. }
```

Destrutores



C++

```
30. // Função Principal
31. int main(int argc, char const *argv[]) {
32.
33.     cout << "Main Function Begins" << endl;
34.     CreateAndDestroy first( 1, "local in main");
35.     CreateAndDestroy second(2, "local in main");
36.     CreateAndDestroy third( 3, "local in main");
37.     CreateAndDestroy forth( 4, "local in main");
38.
39.     return 0;
40. }
41.
```

Destrutores



Python

Destrutores



Python

```
1.  Import copy
2.  class CreateAndDestroy:
3.
4.      def __init__(self, name):
5.          self.__name = name
6.          print(f"Object {self.__name} created.")
7.
8.      def __del__(self):
9.          print(f"Object {self.__name}")
10.
11.     def setName(self, name):
12.         self.__name = name
13.
14.     def getName(self):
15.         return(self.__name)
```

Destrutores



Python

```
16. if __name__ == "__main__":
17.
18.     obj1 = CreateAndDestroy("1")
19.     obj2 = CreateAndDestroy("2")
20.
21.     # Cuidado, copia a referencia (similar a um ponteiro)
22.     obj3 = obj2
23.     obj3.setName("3")
24.
25.     obj4 = copy.copy(obj1) # cria uma cópia do objeto
26.     obj4.setName("4")
27.
28.     # Deletando objetos (é chamado automaticamente)
29.     del obj1
30.     del obj2
31.     #del obj3
32.     del obj4
```

Roteiro



- 1 Introdução
- 2 Construtores
- 3 Destrutores
- 4 Exemplos e Exercícios
- 5 Referências

Exercícios



HANDS ON :)))

Exercício 01

- Crie uma classe **Logger** que, ao ser construída, abre um arquivo de texto para escrita e registra a data/hora de início da sessão. A classe deve ter um método **registrar(mensagem)** que escreve uma linha no arquivo. O **destrutor** deve registrar a hora de encerramento e fechar o arquivo corretamente. O objetivo é garantir que o arquivo nunca fique aberto sem ser fechado, mesmo que o objeto saia de escopo inesperadamente.

Exercício 02

- Crie uma classe **Matriz** que aloca dinamicamente um array bidimensional de inteiros no **construtor**, recebendo linhas e colunas como parâmetros. Implemente o **destrutor** para liberar a memória alocada. Em seguida, implemente o **construtor de cópia** para que duas matrizes não compartilhem o mesmo bloco de memória (**cópia profunda**). Demonstre na função **main** que modificar uma cópia não afeta o original.

Entregável Semanal 03

□ Classe Jogador

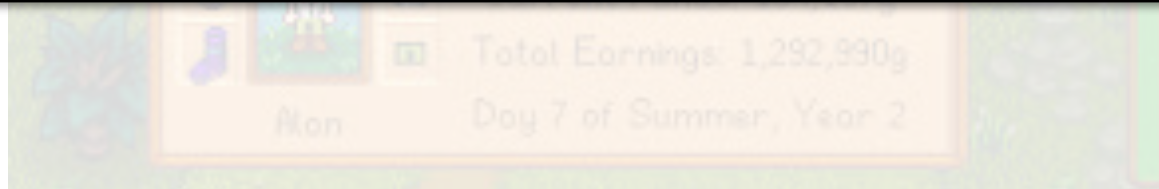


Entregável Semanal 03

□ Classe Jogador

Objetivo: Sistema de progressão baseado em experiência, e jogador com múltiplos construtores

- Criar construtores para inicializar jogadores adequadamente
- Implementar sistema de **experiência (xp)** e **subir_nivel()**
- Gerenciamento de memória: destrutores para liberar recursos



Roteiro



- 1 Introdução
- 2 Visibilidade
- 3 Exemplos: C++ vs Python
- 4 Exercícios
- 5 Referências

Referências sugeridas



[Deitel & Deitel, 2006]

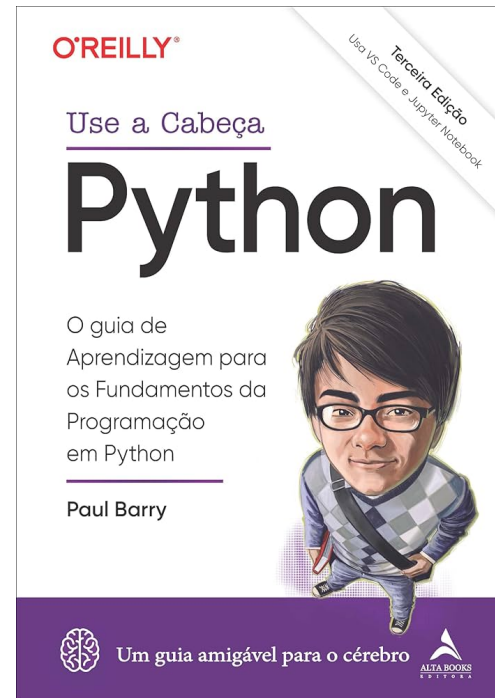


[Meyers, 2016]

Referências sugeridas



[Ziviani, 2007]



[Barry, 2025]



Obrigado :)

Prof. Rafael G. **Mantovani**
rafaelmantovani@utfpr.edu.br